PYTHON FUNDAMENTALS

TOKENS

In a passage of text, individual words and punctuation marks are called tokens. The smallest individual unit in a program is known as Tokens. Python has following tokens:

- ✓ Keywords
- ✓ Identifiers(Name)
- ✓ Literals
- ✓ Operators
- ✓ Punctuators

KEYWORDS

Keywords are the reserved words and have special meaning for python interpreter. Every keyword is assigned specific work and it can be used only for that purpose.

A partial list of keywords in Python is

False	class	finally	import	return
True	continue	for	lambda	try
None	def	from	nonlocal	with
and	del	global	not	while
as	else	if	or	yield
assert	elif	in	pass	
break	except	is	raise	

IDENTIFIERS

Are the names given to different parts of program like variables, objects, classes, functions etc.

Identifier forming rules of Python are :

- Is an arbitrarily long sequence of letters and digits
- The first character must be letter or underscore
- Upper and lower case are different
- The digits 0-9 are allowed except for first character
- It must not be a keyword
- No special characters are allowed other than underscore is allowed.
- Space not allowed

The following are some ***valid*** identifiers

GradePay	File_12_2018	JAMES007
GRADEPAY	_ismarried	_to_update

The following are some ***invalid*** identifiers

Grade-Pay	12_2018_File	$JAMES007
if	RollNo.	Roll No

Literals / Values

- Literals are data items that have a fixed value. Python supports several kinds of literals:
  - String Literal
  - Numeric Literals
  - Boolean Literals
  - Special Literals – ***None***
  - Literal Collections

String Literals

- It is a collection of character(s) enclosed in a double or single quotes
- Examples of String literals
  - “Python”
  - ‘123456'
  - "Hello How are you"
- In Python both single character or multiple characters enclosed in quotes are treated as same

Numeric Literals

- The numeric literals in Python can belong to any of the following numerical types:

  1)<u>Integer Literals:</u> it contain at least one digit and must not contain decimal point. It may contain (+) or (-) sign.

  - Types of Integer Literals:

    a) Decimal : 1234, -50, +100

    b)Octal : it starts from symbol **0o (zero followed by letter 'o')**

    - For e.g. **0o10** represent decimal 8

Numeric Literals

>>> num = **0o10**

>>> print(num)

It will print the value 8

c) Hexadecimal : it starts from **0x (zero followed by letter 'x')**

>>> num = **0xF**

>>> print(num)

it will print the value 15

Numeric Literals

- **2) Floating point Literals:** also known as real literals. Real literals are numbers having fractional parts. It is represented in two forms Fractional Form or Exponent Form
- **Fractional Form:** it is signed or unsigned with decimal point
  - For e.g. 12.0, -15.86, 0.5, 10. (will represent 10.0)
- **Exponent Part:** it consists of two parts "Mantissa" and "Exponent".
  - For e.g. 10.5 can be represented as $0.105 \times 10^2 = 0.105E02$ where 0.105 is mantissa and 02 (after letter E) is exponent

Boolean Literals

A Boolean literals in Python is used to represent one of the two Boolean values i.e. **True or False**

These are the only two values supported for Boolean Literals

For e.g.

```
>>> isMarried=True
>>> type(isMarried)
<class 'bool'>
```

Special Literals *None*

Python has one special literal, which is None. It indicate absence of value. In other languages it is knows as NULL.

```
>>> salary=None
>>> type(salary)
<class 'NoneType'>
```

Complex Numbers

Complex: Complex number in python is made up of two floating point values, one each for real and imaginary part. For accessing different parts of variable (object) x; we will use x.real and x.image. Imaginary part of the number is represented by "j" instead of "I", so 1+0j denotes zero imaginary.

part.

Example

```
>>> x = 1+0j
>>> print x.real,x.imag
1.0 0.0
```

Example

```
>>> y = 9-5j
>>> print y.real, y.imag
9.0 -5.0
```

Simple Input and Output

- In python we can take input from user using the built-in function input().
- Syntax

variable = input(<message to display>)

Note: value taken by input() function will always be of String type, so by default you will not be able to perform any arithmetic operation on variable.

```
>>> marks=input("Enter your marks ")
Enter your marks 100
>>> type(marks)
<class 'str'>
```

Here you can see even we are entering value 100 but it will be treated as string and will not allow any arithmetic operation

Simple Input and Output

```
>>> salary=input("Enter your salary ")
Enter your salary 5000
>>> bonus = salary*20/100
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for /: 'str' and 'int'
```

Reading / Input of Numbers

- Now we are aware that input() function value will always be of string type, but what to do if we want number to be entered. ***The solution to this problem is to convert values of input() to numeric type using int() or float() function.***

Program 1

Open a new script file and type the following code:

```
num1=int(input("Enter Number 1 "))
num2=int(input("Enter Number 2 "))
num3 = num1 + num2
print("Result =",num3)
```

Save and execute by F5 and observe the result

Let us write few programs

- WAP to enter length and breadth and calculate area of rectangle
- WAP to enter radius of circle and calculate area of circle
- WAP to enter Name, marks of 5 subject and calculate total & percentage of student
- WAP to enter distance in feet and convert it into inches
- WAP to enter value of temperature in Fahrenheit and convert it into Celsius.
- WAP to enter radius and height of cylinder and calculate volume of cylinder.

Operators

- are symbol that perform specific operation when applied on variables. Take a look at the expression: **(Operator)**

10 + 25 (Operands)

Above statement is an expression (combination of operator and operands)

i.e. operator operates on operand. some operator requires two operand and some requires only one operand to operate

Types of Operators

- Unary operators: are those operators that require one operand to operate upon. Following are some unary operators:

Operator	Purpose
+	Unary plus
-	Unary minus
~	Bitwise complement
Not	Logical negation

Types of Operators

- Binary Operators: are those operators that require two operand to operate upon. Following are some Binary operators:

1. Arithmetic Operators

Operator	Action
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder
**	Exponent
//	Floor division

Example

```
>>> num1=20
>>> num2=7
>>> val = num1 % num2
>>> print(val)
6
>>> val = 2**4
>>> print(val)
16
>>> val = num1 / num2
>>> print(val)
2.857142857142857
>>> val = num1 // num2
>>> print(val)
2
```

Bitwise operator

Operator	Purpose	Action
&	Bitwise AND	Return 1 if both inputs are 1
^	Bitwise XOR	Return 1, if the number of 1 in input is in odd
|	Bitwise OR	Return 1 if any input is 1

Bitwise operator works on the binary value of number not on the actual value. For example if 5 is passed to these operator it will work on 101 not on 5. Binary of 5 is 101, and return the result in decimal not in binary.

Example

```
>>> print(12 & 7)
4
```

Binary of 12 is 1100 and 7 is 0111, so applying &

1100

0111

0100 Which is equal to decimal value 4

Shift Operators

Operators	Purpose
<<	Shift left
>>	Shift right

Identity Operators

Operators	Purpose
is	Is the Identity same?
is not	Is the identity not same?

Relational Operators

Operators	Purpose
<	Less than
>	Greater than
<=	Less than or Equal to
>=	Greater than or Equal to
==	Equal to
!=	Not equal to

Logical Operators

Operators	Purpose
and	Logical AND
or	Logical OR
not	Logical NOT

Assignment Operators

Operators	Purpose
=	Assignment
/=	Assign quotient
+=	Assign sum
-=	Assign difference
*=	Assign product
**=	Assign Exponent
//=	Assign Floor division

Membership Operators

Operators	Purpose
in	Whether variable in sequence
not in	Whether variable not in sequence

Punctuators

- Punctuators are symbols that are used in programming languages to organize sentence structure, and indicate the rhythm and emphasis of expressions, statements, and program structure.
- **Common punctuators are: ‘ “ # $ @ []{}=:;(),.**

Comments

- **Full line comment**

Example:

```
#This is program of volume of cylinder
```

- **Inline comment**

Example

```
area = length*breadth        # calculating area of rectangle
```

- **Multiline comment**

Example 1 (using #)

```
# Program name: area of circle
# Date: 20/07/18
#Language : Python
```

Comments

- **Multiline comment (using “ “ “) triple quotes**

Example

```
“ “ “
        Program name : swapping of two number
        Date         : 20/07/18
        Logic        : by using third variable
”””
```

Multiple Assignments

- Python is very versatile with assignments. Let's see in how different ways we can use assignment in Python:

1. Assigning same value to multiple variable

```
a = b = c = 50
```

2. Assigning multiple values to multiple variable

```
a,b,c = 11,22,33
```

Note: While assigning values through multiple assignment, remember that Python first evaluates the RHS and then assigns them to LHS

Examples:

Multiple Assignments

```
x,y,z = 10,20,30                #Statement 1
z,y,x = x+1,z+10,y-10           #Statement 2
print(x,y,z)
```

Output will be

10 40 11

Now guess the output of following code fragment

```
x,y = 7,9
y,z = x-2, x+10
print(x,y,z)
```

Multiple Assignments

Let us take another example

y, y = 10, 20

In above code first it will assign 10 to y and again it assign 20 to y, so if you print the value of y it will print ***20***

Now guess the output of following code

```
x, x = 100,200
y,y = x + 100, x +200
print(x,y)
```

Output through print()

- Python allows to display output using print().
- Syntax:

  print(message_to_print[,sep="string",end="string"])

  Example 1

  print("Welcome")

  Example 2

  Age=20

  print("Your age is ", Age)

Output through print()

Example 3

```
r = int(input("Enter Radius "))
print("Area of circle is ",3.14*r*r)
```

Note: from the Example 2 and Example 3 we can observe that while printing numeric value print() convert it into equivalent string and print it. In case of expression (Example 3) it first evaluate it and then convert the result to string before printing.

Output through print()

Note:

print() automatically insert space between different values given in it. The default argument of ***sep*** *parameter of print() is space(' ') .*

Example

print("Name is","Vicky")

Output

Name is Vicky

Output through print()

Note:

Python allows to change the separator string.

Example

print(“Name is”,”Vicky”,”Singh”,sep=“##”)

Output

Name is ##Vicky##Singh

Output through print()

Note:

Be default each print statement print the value to next line. The default value of end is "\n"

Example

print("Learning Python")

print("Developed by Guido Van Rossum")

Output

Learning Python

Developed by Guido Van Rossum

Output through print()

Note:

We can change the value of **end** to any other value.

Example

```
print("Learning Python",end=" ")
print("Developed by Guido Van Rossum")
```

Output

```
Learning Python Developed by Guido Van Rossum
```

Output through print()

Can you Guess the output

```
Name="James"
Salary=20000
Dept="IT"
print("Name is ",Name,end="@")
print(Dept)
print("Salary is ",Salary)
```

Just a minute…

- What is the difference between keywords and identifiers
- What are literals in Python? How many types of literals in python?
- How many types of String in python?

Just a minute…

- What are the different ways to declare multiline String?
- Identify the type of following literal:

  41.678, 12345, True, “False”, 0xCAFE, 0o456, 0o971
- Difference between Expression and Statement

Just a minute…

- The following code is not giving desired output. We want to enter value as 100 and obtain output as 200. Identify error and correct the program

```
num = input("enter any number")
double_num = num * 2
Print("Double of",num,"is",double_num)
```

Just a minute…

- WAP to obtain temperature in Celsius and convert it into Fahrenheit.
- What will be the output of following code:

```
x, y = 6,8
x,y = y, x+2
print(x,y)
```

- What will be the output of following code:

```
x,y = 7,2
x,y,x = x+4, y+6, x+100
print(x,y)
```